

Freelance AI Agent

A full A-to-Z build plan for an automated system to find, score, and apply to freelance jobs & clients worldwide

Important Reality Check

Most freelance platforms (Upwork, Fiverr, Freelancer.com) explicitly prohibit fully automated bidding or proposal submission in their Terms of Service, and they actively detect bot behavior (login patterns, typing cadence, API abuse). Getting an account banned would set you back far more than manual applying.

The recommended design is therefore **not** "fully autonomous apply-while-you-sleep" — it is **"AI does ~95% of the work, you approve/send with one tap."** This still saves enormous time, keeps your accounts safe, and is fully realistic to run 24/7 up to the final submission step. True 24/7 automation is realistic for job discovery, scoring, and drafting — not for final submission on most platforms.

1. Define Scope & Positioning

- Pick 2-3 target platforms based on API / automation feasibility (see table below)
- Define your niche filters: tech stack keywords (Symfony, Laravel, API Platform, React, AI/MCP), budget range, client geography, project type
- Write a positioning document — your value proposition, rate range, portfolio links — this becomes the source data for AI-generated proposals

2. Platform Feasibility Audit

Platform	Automation Feasibility	Notes
Upwork	Low for auto-apply	Limited official API (mostly enterprise/agency), no public "submit proposal" API for individuals. Scraping job feed violates ToS.
Freelancer.com	Medium	Documented public API including project search and bid placement, but bulk automated bidding is monitored.
Fiverr	Very low	No public API for buyer requests; primarily inbound — automation shifts to gig SEO/optimization, not applying.
LinkedIn (jobs/outreach)	Low	Strict anti-automation detection; use official APIs only, no scraping.
Contra, Toptal, We Work Remotely, RemoteOK	Medium-high	Several have RSS/JSON feeds or lax scraping policies — good automation targets.
Direct client outreach (cold email, LinkedIn content)	High	Fully automatable and where most 24/7 effort should go.

Recommendation: build the automation engine platform-agnostic, feed it from (a) official APIs where they exist, (b) RSS/public job feeds, (c) a human-approved semi-auto flow for Upwork/Fiverr where the agent drafts and you click send.

3. System Architecture

The pipeline below is fully automatable end-to-end up through the review queue — ingestion, scoring, and drafting never sleep. Submission is where human approval is inserted to stay compliant with platform ToS.

Stage	Function
Job Sources	APIs, RSS feeds, public job boards
Ingestion Worker	Polls sources every N minutes, dedupes, stores raw postings
Scoring / Filter (Claude API)	LLM scores job fit — skills match, budget, red flags (1-10)
Proposal Generator (Claude API + RAG)	RAG over your portfolio/past proposals, generates tailored draft
Review Queue (human-in-loop)	Telegram/Slack bot sends job summary + draft with Approve / Edit / Skip buttons
Submission Layer	API call where available; otherwise you click send
CRM / Tracker (Postgres + dashboard)	Logs every application, status, follow-ups, win rate

4. Detailed Build Steps

Step 1 — Ingestion

Python workers (Celery + Redis) polling each source on a schedule. Store raw postings in Postgres with a source, raw_text, posted_at, status schema.

Step 2 — Scoring

Prompt the Claude API with your positioning doc + job text, returning structured JSON: fit_score, missing_skills, red_flags, suggested_rate. Auto-discard anything below your threshold (e.g. under 6/10) without bothering you.

Step 3 — Proposal Generation (RAG)

Build a small vector store (Chroma or pgvector, since Postgres already supports it) of your past winning proposals, case studies, and portfolio pieces. Retrieve relevant snippets per job, feed to Claude API to draft a proposal in your voice. Keep 5-10 real proposals as few-shot style anchors.

Step 4 — Review / Approval Interface

Simplest option: a Telegram bot (python-telegram-bot) sending cards with inline buttons. Nicer option: a small React dashboard showing a queue with edit-in-place before sending. This step keeps you ToS-compliant and your accounts safe.

Step 5 — Submission

Freelancer.com: use their official bid API directly. Upwork/Fiverr: click send manually from the drafted text.

Direct outreach (cold email to agencies/companies): fully automatable via SMTP with sending-limit throttling to avoid spam flags.

Step 6 — CRM & Follow-up

Track: applied → viewed → replied → interview → won/lost. Auto-schedule follow-up reminders (e.g. no reply in 5 days → draft follow-up). Weekly summary report on applications sent, response rate, and win rate feeds back into refining the scoring prompt.

Step 7 — Deployment (24/7)

Small VPS (2-4GB RAM is enough) with Docker Compose services for ingestion worker, scoring worker, bot, and Postgres. Celery beat for scheduling. Add uptime monitoring (Healthchecks.io free tier) so you know if a worker dies silently.

5. Tools & Software List

Core Agent Logic

- Claude API (Sonnet — for drafting and scoring, cost-effective at this volume)
- Python + raw Anthropic SDK (simpler than a framework for this use case)

Infrastructure

- Docker + Docker Compose
- Hetzner or existing VPS
- Postgres with pgvector extension (for RAG — avoids a separate vector DB)
- Redis + Celery for job queues and scheduling

Data Ingestion

- requests / httpx for API sources
- feedparser for RSS job feeds
- Playwright — only for ToS-permitted scraping/semi-auto fill, never to bypass bot detection or CAPTCHAs

Notifications / Approval UI

- Telegram Bot API (fastest to build), or a small React + FastAPI dashboard
- n8n (self-hosted workflow tool) as an option to replace ingestion/approval/CRM glue with less custom code

Outreach

- Transactional email service (Postmark, SES) for direct client outreach — better deliverability than raw SMTP
- LinkedIn: official tools only (e.g. Sales Navigator's own export) — avoid automated LinkedIn actions entirely, detection is aggressive and bans are hard to reverse

Monitoring

- Healthchecks.io or UptimeRobot (free tiers) to alert if a worker stalls

6. Rough Build Order (6-Week Sprint Plan)

Week	Focus
Week 1	Postgres schema + ingestion from 2-3 sources (start with Freelancer API + one RSS feed)
Week 2	Scoring prompt + filtering logic
Week 3	RAG proposal generator + your proposal library

Week 4	Telegram approval bot + submission flow
Week 5	CRM dashboard + follow-up automation
Week 6	Deploy, monitor, and tune based on real response data

Prepared as a working reference document — architecture and tools chosen to fit a Python / Docker / Postgres / Celery stack.